TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Topic – DBMS

Lecture No.- 12

By- Rahul Sir

Topics to be Covered

Topic

1:- Functional Dependency
2:- Types of Functional Dependency
3:- Trivial Function Dependency
4:- Non-Trivial Functional Dependency
5-Multivalued Functional Dependency
6- Transitive Functional Dependency
7- Full and Partial Functional Dependency
8- Conclusion.
9->2-Normal Form
10-> 3-Normal Form

Types of Functional Dependencies in DBMS

1. Trivial functional dependency
2. Non-Trivial functional dependency
3. Multivalued functional dependency
4. Transitive functional dependency

1. Trivial Functional Dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e. If X → Y and Y is the subset of X, then it is called trivial functional dependency.

Symbolically: A→B is trivial functional dependency if B is a subset of A.

The following dependencies are also trivial: A→A & B→B

$x \subseteq y$

$(A,B) \rightarrow \cancel{B}A$

B

AB

Example 1 :

- ABC -> AB
- ABC -> A
- ABC -> ABC

Topic :Trivial Functional Dependency

Example 1 :

- ABC -> AB
- ABC -> A
- ABC -> ABC

Example 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, {roll_no, name} → name is a trivial functional dependency, since the dependent name is a subset of determinant set {roll_no, name}. Similarly, roll_no → roll_no is also an example of trivial functional dependency.

$AB \to C$

2. Non-trivial Functional Dependency

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant. i.e. If **X → Y** and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

trivial

$\{x, y\} \to x$, y, xy, z

$x \notin y$

$\{x, y\} \to z$

Non Trivial

Example 1 :

- Id -> Name
- Name -> DOB

Topic :Non-Trivial Functional Dependency

Example 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, roll_no → name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no. Similarly, {roll_no, name} → age is also a non-trivial functional dependency, since age is not a subset of {roll_no, name}

Topic :Multivalued Functional Dependency

TEACHING WALLAH

4. Multivalued Functional Dependency

$AB \rightarrow DC$ $\quad x \rightarrow y$

In Multivalued functional dependency, entities of the dependent set are not dependent on each other. i.e. If a → {b, c} and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.

Example:

determinis Attribute $a \rightarrow \{b, c\}$ $\quad \{id\} \leftarrow \{course\}$

A bike_model	B manuf_year Man_Year	C color Colors
tu1001	2007	Black
tu1001	2007	Red
tu2012	2008	Black
tu2012	2008	Red

id	name	course
1	Raju	C++
2	Rohan	C
3	Mohan	Java

dependent Attribute

$\{id, name, course\} \leftarrow \{course$

Superkey

name
id
course

Topic :Multivalued Functional Dependency

In this table:

- X: bike_model
- Y: color
- Z: manuf_year

{bike model no} ← {colour, manyear}

bike model no	colour, manyear
RTR	white, 2024
RTR	yellow, 2025

For each bike model (bike_model):

1. There is a group of colors (color) and a group of manufacturing years (manuf_year).
2. The colors do not depend on the manufacturing year, and the manufacturing year does not depend on the colors. They are independent.
3. The sets of color and manuf_year are linked only to bike_model.

That's what makes it a multivalued dependency.

In this case these two columns are said to be multivalued dependent on bike_model.
These dependencies can be represented like this:

Topic :Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If a → b & b → c, then according to axiom of transitivity, a → c. This is a transitive functional dependency.

X = Candidate Key
X = Prime Attribute

Example:

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

$X \leftarrow Y \leftarrow Z$

P NP NP NP

$x \leftarrow Z$

3NF

$x \rightarrow y$

$x \rightarrow z$

~~3NF~~

$y \leftarrow z$

NP NP

Here, enrol_no → dept and dept → building_no. Hence, according to the axiom of transitivity, enrol_no → building_no is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

Topic :Full and Partial Functional Dependency

6. Fully Functional Dependency

Id	course	fee
1	Java	1000
2	C	500
1	Python	1500

$\{x, y\} \longrightarrow z$

$x \rightarrow z$

In full functional dependency an attribute or a set of attributes uniquely determines another attribute or set of attributes. If a relation R has attributes X, Y, Z with the dependencies X->Y and X->Z which states that those dependencies are fully functional.

7. Partial Functional Dependency

$\underbrace{\{x, y\}}_{\text{Composite key}} \longrightarrow z$

In partial functional dependency a non key attribute depends on a part of the composite key, rather than the whole key. If a relation R has attributes X, Y, Z where X and Y are the composite key and Z is non key attribute. Then X->Z is a partial functional dependency in RBDMS.

Topic :Conclusion on Functional Dependency

TEACHING WALLAH

Conclusion

Functional dependency is very important concept in database management system for ensuring the data consistency and accuracy.

$\{A\,B\,C\} \rightarrow \{B\}$

What is Second Normal Form (2NF)?

Second Normal Form (2NF) is based on the concept of fully **functional dependency**. It is a way to organize a database table so that it reduces **redundancy** and ensures **data consistency**. For a table to be in 2NF, it must first meet the requirements of **First Normal Form** (1NF), meaning all columns should contain **single**, indivisible values without any repeating groups. Additionally, the table should not have **partial dependencies**.

The primary goal of **Second Normal Form** is to eliminate **partial dependencies**. A **partial dependency** happens when a non-prime attribute (an attribute not part of a candidate key) depends on only a part of a composite primary key, rather than on the entire key. Removing these partial dependencies helps in reducing redundancy and preventing update anomalies.

Example of Second Normal Form (2NF)

Consider a table storing information about students, courses, and their fees:

COURSES Table

STUD_NO	COURSE_NO	COURSE_FEE
1	C1	1000
2	C2	1500
1	C4	2000
4	C3	1000
4	C1	1000
2	C5	2000

id → Name

PA NPA

- There are many courses having the same course fee. Here, COURSE_FEE cannot alone decide the value of COURSE_NO or STUD_NO.
- COURSE_FEE together with STUD_NO cannot decide the value of COURSE_NO.
- COURSE_FEE together with COURSE_NO cannot decide the value of STUD_NO.
- The candidate key for this table is `{STUD_NO, COURSE_NO}` because the combination of these two columns uniquely identifies each row in the table.
- COURSE_FEE is a **non-prime attribute** because it is not part of the candidate key `{STUD_NO, COURSE_NO}`.
- But, COURSE_NO -> COURSE_FEE, i.e., COURSE_FEE is dependent on COURSE_NO, which is a proper subset of the candidate key.
- Therefore, Non-prime attribute COURSE_FEE is dependent on a proper subset of the candidate key, which is a partial dependency and so this relation is not in 2NF.

Topic : 2-NF

TEACHING WALLAH

STUDENT_COURSES Table

STUD_NO	COURSE_NO
1	C1
2	C2
1	C4
4	C3
4	C1
2	C5

FK

COURSES Table

COURSE_NO	COURSE_FEE
C1	1000
C2	1500
C4	2000
C3	1000
C5	2000

{studno, cours} course fee

Now, each table is in 2NF:

- The Course Table ensures that **COURSE_FEE** depends only on **COURSE_NO**.
- The Student-Course Table ensures there are no partial dependencies because it only relates students to courses.

Now, the **COURSE_FEE** is no longer repeated in every row, and each table is free from partial dependencies. This makes the database **more efficient** and **easier to maintain**.

Topic : 2-NF

TEACHING WALLAH

$E \leftarrow F$

Question:- R(A,B,C,D,E,F)

FD:- { C→F, E→A, EC→ D, A→B}

p Np

I- What is candidate key ?

$\{CE\}^+ \rightarrow \{F, A, D, B, C, E\}$

2- Which Attribute is Prime Attribute ? → E, C

F

3- Which Attribute is Non Prime Attribute ?

4- Check in Relations There is Partial Dependency or not {A, B, D, F}

5- Check this Relation or table is in 2-NF or not

✗

Topic : 2-NF

RULE TO CHECK PARTIAL DEPENDENCY

$EC^+ = \{ECFADB\}$

$CK = \{EC\}$

PRIME ATTRIBUTE= {E,C}

NON PRIME ATTRIBUTE= {A,B,D,F}

$\boxed{C} \leftarrow F$

Proper Subset Of CK   NP

{A B}

Determinate attribute should be proper subset of Candidate key

&

$\{E, C\} \rightarrow F$

Dependent attribute should be a Non Prime Attribute

It Implies that→ {L.H.S should be proper subset of Candidate key

&

R.H.S Should be a Non Prime attribute

Topic : 3-NF

TEACHING WALLAH

Normalization in database design is an important process for organizing data to **reduce redundancy**, maintain **data integrity** and improve accuracy. The **Third Normal Form (3NF)** builds on the First (1NF) and Second (2NF) Normal Forms. Achieving 3NF ensures that the database structure is free of transitive dependencies, reducing the chances of data anomalies.

Even though tables in 2NF have reduced redundancy compared to 1NF, they may still encounter issues like **update anomalies**. For example, if one row is updated and another one is not, this can lead to inconsistent data. This happens due to **transitive dependencies**, which 3NF resolves by removing such dependencies, making the database more reliable.

Topic : 3-NF

TEACHING WALLAH

What is Third Normal Form (3NF)?

A relation is in **Third Normal Form (3NF)** if it satisfies the following two conditions:

1. It is in **Second Normal Form (2NF)**: This means the table has no partial dependencies (i.e., no non-prime attribute is dependent on a part of a candidate key).
2. There is **no transitive dependency** for **non-prime attributes**: In simpler terms, no non-key attribute should depend on another non-key attribute. Instead, all non-key attributes should depend directly on the primary key.

Understanding Transitive Dependency

To fully grasp 3NF, it's essential to understand **transitive dependency**. A **transitive dependency** occurs when one non-prime attribute depends on another non-prime attribute rather than depending directly on the primary key. This can create redundancy and inconsistencies in the database.

For example, if we have the following relationship between attributes:

- **A -> B** (A determines B)
- **B -> C** (B determines C)

Topic : 3-NF

TEACHING WALLAH

Roll_no	State	city
1	U.P	MIRZAPUR
2	BIHAR	PATNA
3	U.P	MATHURA
4	PUNJAB	JALANDHAR
5	U.P	MATHURA

ROLL_NO→ STATE STATE→CITY

TRANSITIVE DEPENDENCY

ROLL_NO→ CITY

Topic : 3-NF

R(A,B,C,D)

FD:- AB->C , C->D

I- Find candidate key

II- Find prime and non prime attribute

III- Check whether it is Transitive dependency or not

Topic : 3-NF

R(A,B,C,D)

FD:- AB->CD , D->A

I- Find candidate key

II- Find prime and non prime attribute

III- Check whether it is Transitive dependency or not

Topic : 3-NF

$AB^+ = ABCD$

$DB^+ = DBAC$

Candidate key:- {AB, DB}

PRIME ATTRIBUTE={A,B,D}

NON PRIME ATTRIBUTE={C}

RULE FOR CHECK 3-NF If relation is already follow 2-NF

= L.H.S Must be a Candidate key or super key

OR

R.H.S is a prime attribute

THANK YOU